

Catálogo de productos con Bootstrap, CSS y JavaScript

Temas: containers, grid, cards, botones, badges, collapse, colores, imágenes, tablas, paginación, CSS personalizado y DOM.

Enunciado

Construye una página web llamada TecnoCatálogo para mostrar un catálogo de productos tecnológicos. La aplicación debe permitir mostrar productos en tarjetas, buscarlos, filtrarlos por categoría, consultar detalles mediante collapse, cambiar entre vista de tarjetas y vista de tabla y navegar mediante paginación.

Estructura del proyecto

```
tecnocatalogo/
├── index.html
├── estilos.css
├── app.js
└── img/
    ├── laptop.jpg
    ├── teclado.jpg
    ├── mouse.jpg
    └── audifonos.jpg
```

Datos de los productos

Crea un arreglo con al menos 8 productos. Cada producto debe contener id, nombre, categoría, precio, estado, imagen y descripción.

```
const productos = [
  {
    id: 1,
    nombre: "Laptop Pro",
    categoria: "Computadoras",
    precio: 850,
    estado: "Disponible",
    imagen: "img/laptop.jpg",
    descripcion: "Laptop para estudio, programación y trabajo."
  },
  {
    id: 2,
    nombre: "Teclado mecánico",
    categoria: "Accesorios",
    precio: 55,
    estado: "Disponible",
    imagen: "img/teclado.jpg",
    descripcion: "Teclado mecánico con iluminación."
  }
];
```

Categorías sugeridas: Computadoras, Accesorios, Audio y Monitores.

Estados: Disponible, Agotado y Oferta.

Encabezado y contenedores

Crea un encabezado de ancho completo y un contenido principal centrado.

```
<header class="container-fluid bg-dark text-white py-5">
  <!-- Nombre de la tienda, descripción y badge -->
</header>

<main class="container py-5">
  <!-- Filtros, tarjetas, tabla y paginación -->
</main>
```

- El encabezado debe mostrar el nombre de la tienda.
- Incluye una descripción breve.
- Agrega un badge con el total de productos.

Filtros

Crea una fila responsive con un buscador, un select de categoría y botones para limpiar filtros y cambiar de vista.

```
<div class="row g-3 mb-4">
  <div class="col-12 col-md-5">
    <!-- Buscador -->
  </div>

  <div class="col-12 col-md-3">
    <!-- Categoría -->
  </div>

  <div class="col-12 col-md-4">
    <!-- Botones -->
  </div>
</div>

<input
  type="search"
  id="buscador"
  class="form-control"
  placeholder="Buscar producto"
>

<select id="filtroCategoría" class="form-select">
  <option value="todas">Todas las categorías</option>
  <option value="Computadoras">Computadoras</option>
  <option value="Accesorios">Accesorios</option>
  <option value="Audio">Audio</option>
  <option value="Monitores">Monitores</option>
</select>
```

Vista de tarjetas

El contenedor debe utilizar el sistema Grid de Bootstrap.

```
<div id="contenedorTarjetas" class="row g-4"></div>
```

Cada tarjeta debe generarse desde JavaScript y usar la siguiente distribución:

```
<div class="col-12 col-md-6 col-lg-4">
  <!-- Card -->
</div>
```

- Una tarjeta por fila en celular.
- Dos tarjetas por fila en tablet.
- Tres tarjetas por fila en computadora.

Cada tarjeta debe mostrar imagen, nombre, categoría, precio, badge de estado, botón de detalles y descripción oculta mediante collapse.

```
<div class="col-12 col-md-6 col-lg-4">
  <article class="card h-100 shadow-sm producto-card">
    

    <div class="card-body d-flex flex-column">
      <h2 class="card-title h5">Laptop Pro</h2>
      <p class="text-secondary">Computadoras</p>

      <span class="badge text-bg-success">
        Disponible
      </span>

      <p class="precio mt-3">$850.00</p>

      <button
        class="btn btn-primary mt-auto"
        data-bs-toggle="collapse"
        data-bs-target="#detalle-1"
      >
        Ver detalles
      </button>

      <div class="collapse mt-3" id="detalle-1">
        <div class="card card-body">
          Laptop para estudio, programación y trabajo.
        </div>
      </div>
    </div>
  </article>
</div>
```

Badges de estado

Crea una función que devuelva la clase Bootstrap correspondiente al estado.

```
function obtenerClaseEstado(estado) {  
  if (estado === "Disponible") {  
    return "text-bg-success";  
  }  
  
  if (estado === "Agotado") {  
    return "text-bg-danger";  
  }  
  
  return "text-bg-warning";  
}
```

Búsqueda

El buscador debe filtrar por nombre, categoría o estado mientras el usuario escribe.

```
buscador.addEventListener("input", aplicarFiltros);
```

Pista para normalizar el texto:

```
const texto = buscador.value  
  .trim()  
  .toLowerCase();
```

Pista para filtrar:

```
const resultados = productos.filter(producto =>  
  producto.nombre.toLowerCase().includes(texto)  
);
```

10. Filtro por categoría

El select debe trabajar al mismo tiempo que el buscador.

```
filtroCategoria.addEventListener(  
  "change",  
  aplicarFiltros  
);
```

La opción todas debe mostrar todos los productos.

Vista de tabla

Crea una tabla responsive y genera sus filas desde JavaScript.

```
<div id="contenedorTabla" class="table-responsive d-none">
  <table class="table table-striped table-hover align-middle">
    <thead class="table-dark">
      <tr>
        <th>Imagen</th>
        <th>Producto</th>
        <th>Categoría</th>
        <th>Precio</th>
        <th>Estado</th>
      </tr>
    </thead>
    <tbody id="cuerpoTabla"></tbody>
  </table>
</div>
```

CSS sugerido para las imágenes pequeñas:

```
.imagen-tabla {
  width: 65px;
  height: 65px;
  object-fit: cover;
}
```

Cambio de vista

```
<button id="botonTarjetas" class="btn btn-primary">
  Tarjetas
</button>

<button id="botonTabla" class="btn btn-outline-primary">
  Tabla
</button>

botonTarjetas.addEventListener("click", () => {
  contenedorTarjetas.classList.remove("d-none");
  contenedorTabla.classList.add("d-none");
});

botonTabla.addEventListener("click", () => {
  contenedorTarjetas.classList.add("d-none");
  contenedorTabla.classList.remove("d-none");
});
```

Paginación

La aplicación debe mostrar 4 productos por página.

```
const productosPorPagina = 4;  
let paginaActual = 1;
```

Calcula el rango de productos que debe mostrarse:

```
const inicio =  
  (paginaActual - 1) * productosPorPagina;  
  
const fin =  
  inicio + productosPorPagina;  
  
const productosPagina =  
  productosFiltrados.slice(inicio, fin);
```

Contenedor HTML de la paginación:

```
<nav class="mt-5" aria-label="Paginación">  
  <ul  
    id="paginacion"  
    class="pagination justify-content-center"  
  ></ul>  
</nav>
```

Cada página debe generarse con JavaScript usando la estructura:

```
<li class="page-item active">  
  <button class="page-link">1</button>  
</li>
```

CSS personalizado

```
body {
  min-height: 100vh;
  background-color: #f4f6f8;
}

.encabezado {
  background:
    linear-gradient(
      rgba(0, 0, 0, 0.75),
      rgba(0, 0, 0, 0.75)
    ),
    url("img/prueba.jpg") center / cover;
}

.producto-card {
  overflow: hidden;
  transition:
    transform 0.2s ease,
    box-shadow 0.2s ease;
}

.producto-card:hover {
  transform: translateY(-6px);
}

.card-img-top {
  height: 220px;
  object-fit: cover;
}

.precio {
  color: #6f42c1;
  font-size: 1.4rem;
  font-weight: bold;
}

.imagen-tabla {
  width: 65px;
  height: 65px;
  object-fit: cover;
  border-radius: 8px;
}
```

15. Funciones obligatorias

```
function aplicarFiltros() {  
}  
  
function renderizarContenido() {  
}  
  
function renderizarTarjetas(lista) {  
}  
  
function renderizarTabla(lista) {  
}  
  
function renderizarPaginacion(totalProductos) {  
}  
  
function cambiarPagina(numeroPagina) {  
}  
  
function obtenerClaseEstado(estado) {  
}  
  
function limpiarFiltros() {  
}
```